ADAN8888 — Week 7: Develop Second Modeling Approach

===== WEEK 7 START =====

Project: Next-month U.S. domestic scheduled-passenger demand for directional carrier-route-month series, based on BTS T-100.

Second method: `HistGradientBoostingRegressor`, compared with Week 6's simpler Ridge regression. The new method captures nonlinear, threshold-based relationships in the two Week 5 PCA inputs. Its complexity is controlled by boosting iterations and tree leaf count.

Week 7 compliance: Exactly **three** preset hyperparameter variations; training and validation MAE/RMSE/R^2 for each; provisional winner chosen by **validation MAE**, with validation RMSE then simpler complexity as tie-breakers.

Execution: Uses `Src/week7_src_hist_gradient_boosting.py` and the same Week 6/Week 5 data loader, running on the full original training and validation partitions. Retains the Week 3 split. Reproduces the same numeric transformations fitted on **training only**.

1. Imports and source location

- Python module contains entire model-evaluation implementation.
- This notebook calls that module rather than replicating its logic.

In [1]:
```
from pathlib import Path
import importlib.util
import sys
import numpy as np
from IPython.display import display, Image

def locate_root():
    here=Path.cwd().resolve()
    for folder in (here, *here.parents):
        if (folder/'Src'/'week7_src_hist_gradient_boosting.py').is_file():
            return folder
    raise FileNotFoundError('Open the ADAN8888 project root or its Notebooks folder.')

ROOT=locate_root()
file=ROOT/'Src'/'week7_src_hist_gradient_boosting.py'
spec=importlib.util.spec_from_file_location('week7_hist_boost',file)
week7=importlib.util.module_from_spec(spec)
sys.modules[spec.name]=week7
spec.loader.exec_module(week7)
REPORTS=ROOT/'Reports'/'week7'
TABLES=REPORTS/'week7_tables'
FIGURES=REPORTS/'week7_figures'
print('Project root:',ROOT)
print('Week 7 source:',file)
print('scikit-learn candidates:',len(week7.CANDIDATES))
```

```
Project root:
Week 7 source:

scikit-learn candidates: 3
```

2. Input data and chronology checks

In [2]:
```
partitions,split_summary,source=week7.load_training_and_validation(ROOT)
display(split_summary)
print('Prior-week pipeline:',source)
assert len(partitions['Training'])==int(split_summary.loc[split_summary.Split.eq('Training'),'Rows'].iloc[0])
assert len(partitions['Validation'])==int(split_summary.loc[split_summary.Split.eq('Validation'),'Rows'].iloc[0])
assert partitions['Training'].ForecastMonth.max() < partitions['Validation'].ForecastMonth.min()
assert all(col in partitions['Training'] for col in ['PC1','PC2','TargetPassengersNextMonth'])
print('Exact chronological separation verified. Test partition is not scored.')
```

	Split	Rows	ForecastMonths	FirstReferenceMonth	LastReferenceMonth	FirstForecastMonth	LastForecastMonth	RowPercent	Mo
0	Training	1062603	95	2015-01-01	2022-11-01	2015-02-01	2022-12-01	67.842078	
1	Validation	239395	20	2022-12-01	2024-07-01	2023-01-01	2024-08-01	15.284216	
2	Test	264291	21	2024-08-01	2026-04-01	2024-09-01	2026-05-01	16.873706	

```
Prior-week pipeline: Week 5 source (reconstructed full PCA from official BTS ZIPs)
Exact chronological separation verified. Test partition is not scored.
```

3. Define exactly three complexity variations

Model	learning_rate	max_iter	max_leaf_nodes	Controlled difference
HGB_A	0.05	100	15	Lower-complexity starting point
HGB_B	0.05	250	15	Isolates longer boosting relative to A
HGB_C	0.05	250	31	Isolates greater tree-leaf capacity relative to B

`min_samples_leaf=50`, `l2_regularization=1`, `random_state=42` and `early_stopping=False` are fixed. Disabling automatic early stopping avoids introducing a random internal 10% validation holdout. Histogram binning and L2 regularization help manage computational cost and complexity. This three-setting design meets the assignment more directly than the eight combinations in the original draft.

In [3]:
```
display(week7.pd.DataFrame(week7.CANDIDATES))
print('Other parameters:',week7.COMMON)
assert len(week7.CANDIDATES)==3
```

	Model	learning_rate	max_iter	max_leaf_nodes
0	HGB_A	0.05	100	15
1	HGB_B	0.05	250	15
2	HGB_C	0.05	250	31

```
Other parameters: {'loss': 'squared_error', 'min_samples_leaf': 50, 'l2_regularization': 1.0, 'early_stopping': False, 'random_
state': 42}
```

4. Train three models; evaluate both partitions

Training provides model-fitting data. The later validation set determines the winner and is not fitted. Models are evaluated with passenger-scale MAE (primary; lower is better), RMSE (emphasizes large misses), and R^2 (relative mean baseline). Negative predictions are clipped to zero for all variations, consistent with Week 6.

Note: The full BTS data pass may require substantial compute. Avoid running the notebook and standalone source back-to-back; either runs the same analysis.

In [4]:
```
results=week7.fit_three_variations(
    partitions['Training'],partitions['Validation'],split_summary,source
)
display(results.comparison.round(4))
assert len(results.comparison)==3
assert len(results.metrics_long)==6
assert results.metrics_long.Split.unique().tolist()==['Training','Validation']
```

```
Completed HGB_A: validation MAE 1,179.31 passengers
Completed HGB_B: validation MAE 1,163.47 passengers
Completed HGB_C: validation MAE 1,161.07 passengers
```

	Model	learning_rate	max_iter	max_leaf_nodes	FittedIterations	Training_MAE	Training_RMSE	Training_R2	Validation_MAE	Validation_R
0	HGB_A	0.05	100	15	100	1180.2188	2263.1843	0.9149	1179.3146	2060
1	HGB_B	0.05	250	15	250	1164.0830	2254.8631	0.9155	1163.4698	2050
2	HGB_C	0.05	250	31	250	1159.0169	2247.9040	0.9160	1161.0682	2051

5. Examine training/validation differences and choose winner

The winner is the candidate with the lowest **validation MAE**, breaking exact ties using validation RMSE and then lower complexity. A larger train–validation gap can reflect complexity-related overfitting, but time-dependent shifts may also change error rates. It is not a proof of overfitting by itself.

In [5]:
```
display(results.ranking[['Model','learning_rate','max_iter','max_leaf_nodes',
                         'Validation_MAE','Validation_RMSE','Validation_R2',
                         'MAE_Gap_ValidationMinusTraining']].round(4))
print('Provisional winner:',results.winning_model_name)
print('Winning settings:',results.winning_settings)
assert results.winning_model_name==results.ranking.iloc[0].Model
```

	Model	learning_rate	max_iter	max_leaf_nodes	Validation_MAE	Validation_RMSE	Validation_R2	MAE_Gap_ValidationMinusTraining
0	HGB_C	0.05	250	31	1161.0682	2051.7879	0.9360	2.0513
1	HGB_B	0.05	250	15	1163.4698	2050.4139	0.9361	-0.6132
2	HGB_A	0.05	100	15	1179.3146	2060.2160	0.9355	-0.9042

```
Provisional winner: HGB_C
Winning settings: {'Model': 'HGB_C', 'learning_rate': 0.05, 'max_iter': 250, 'max_leaf_nodes': 31}
```

6. Export reproducible outputs

- Save the three-model comparison, six row-wise score records, validation ranking, monthly validation MAE, selected-model manifest and joblib estimator.
- All fitted parameters preserved in Week 5 source/transform metadata and the selected Week 7 manifest.

In [9]:
```
week7.save_week7(results,ROOT)
print('Report tables:',[p.name for p in sorted(TABLES.glob('*'))])
print('Saved model:',ROOT/'Models'/'week7'/'week7_selected_hist_gradient_boosting.joblib')
print((REPORTS/'week7_verified_results_for_report.md').read_text(encoding='utf-8'))
```

```
Report tables: ['week7_metrics_long.csv', 'week7_monthly_validation_mae.csv', 'week7_split_summary.csv', 'week7_three_model_com
parison.csv', 'week7_validation_ranking.csv', 'week7_winner_summary.json']
Saved model:

# Week 7 — Empirical Results (See written report)

Values generated by executing Week 7 pipeline.

Selected: **HGB_C**, hyperparameters: {'learning_rate': 0.05, 'max_iter': 250, 'max_leaf_nodes': 31}
Validation MAE: **1,161.07** passengers. The held-out test set was NOT evaluated.

| Model | Learning rate | Iterations | Leaf nodes | Train MAE | Val MAE | Train RMSE | Val RMSE | Train R² | Val R² |
|---|---|---|---|---|---|---|---|---|---|
| HGB_A | 0.05 | 100 | 15 | 1,180.22 | 1,179.31 | 2,263.18 | 2,060.22 | 0.9149 | 0.9355 |
| HGB_B | 0.05 | 250 | 15 | 1,164.08 | 1,163.47 | 2,254.86 | 2,050.41 | 0.9155 | 0.9361 |
| HGB_C | 0.05 | 250 | 31 | 1,159.02 | 1,161.07 | 2,247.90 | 2,051.79 | 0.9160 | 0.9360 |

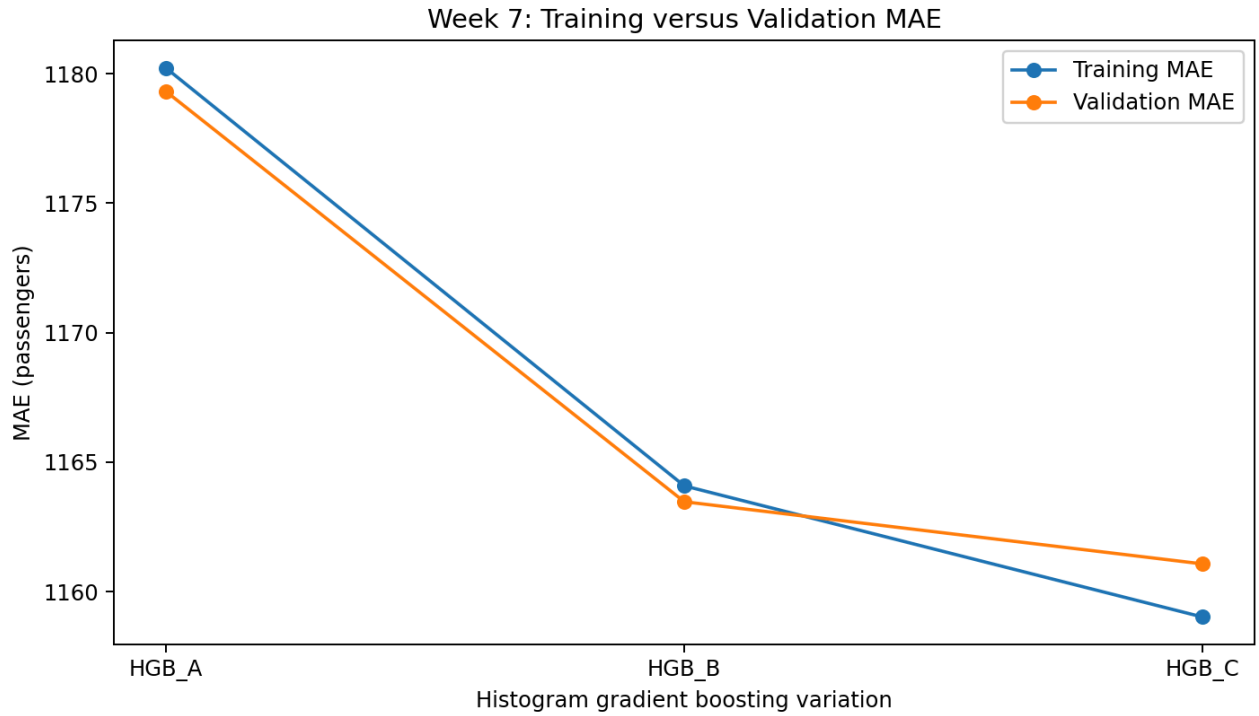
See `week7_figures/` for generated figures.
```

7. Review graphs and forecast error behavior

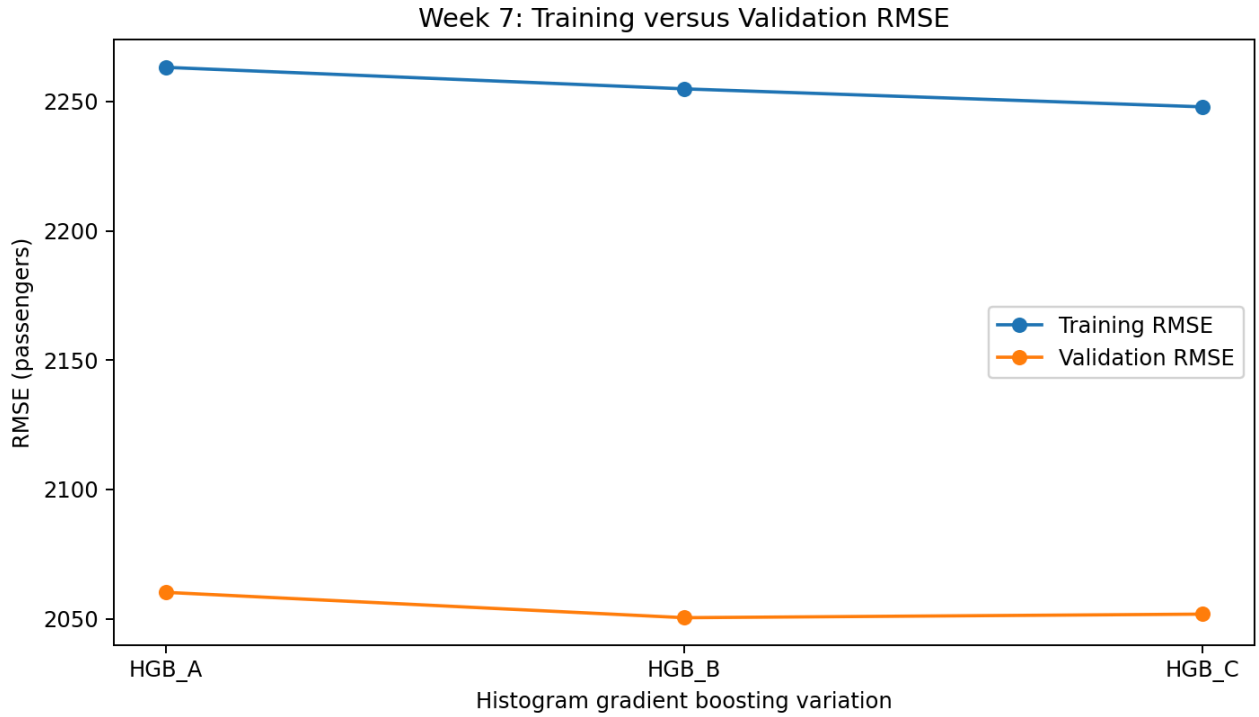
The first two plots compare full-partition MAE and RMSE. The actual-vs-predicted scatter uses a reproducible 4,000-row sample **only for visualization**; metrics use all rows. Monthly validation MAE uses the complete validation partition.

In [10]:
```
for filename in [
    'week7_figure_mae_by_variation.png',
    'week7_figure_rmse_by_variation.png',
    'week7_figure_validation_actual_vs_predicted.png',
    'week7_figure_monthly_validation_mae.png',
]:
    chart=FIGURES/filename
    assert chart.is_file(),f'Missing chart: {chart}'
    print(filename)
    display(Image(filename=str(chart)))
```

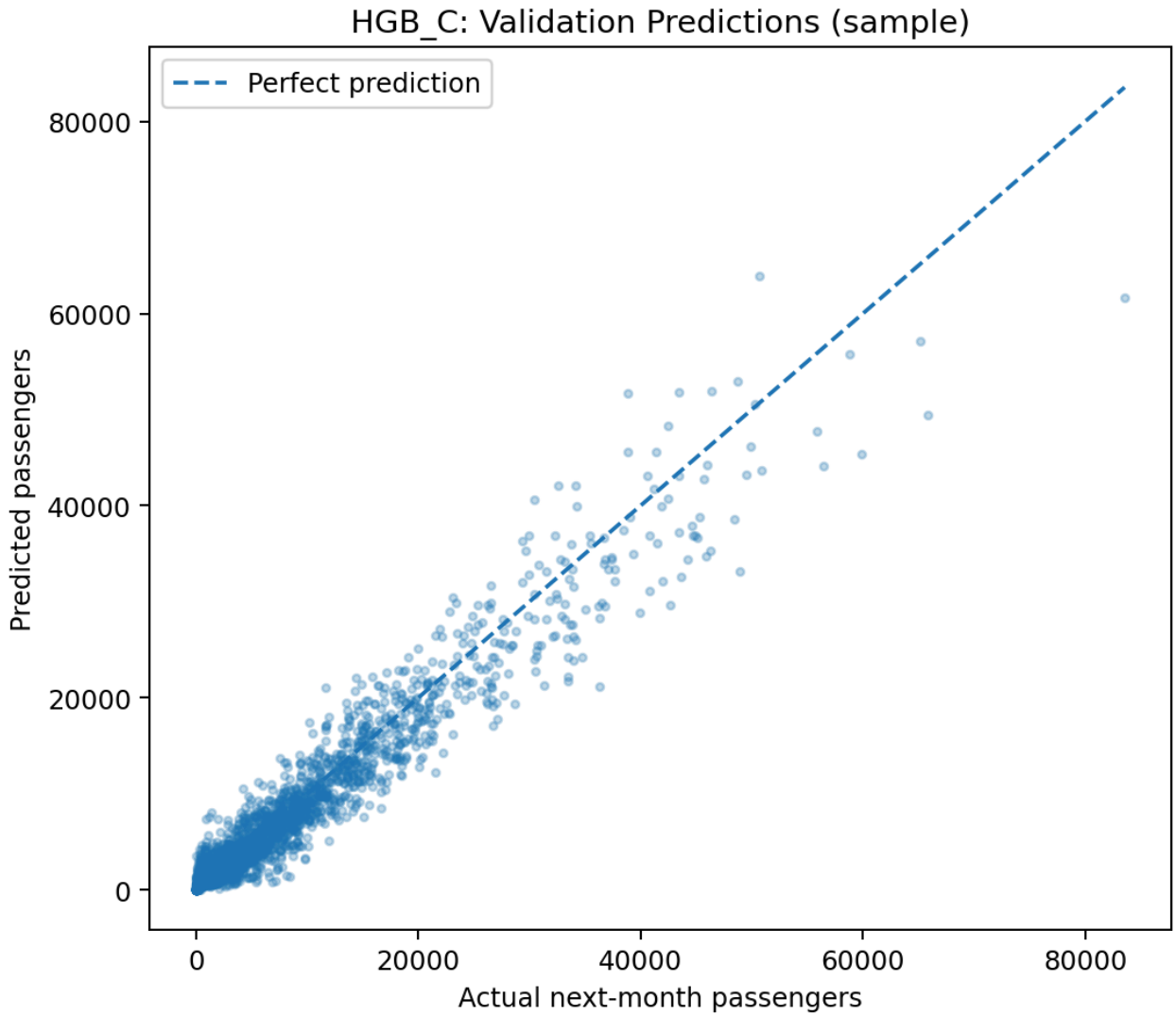
```
week7_figure_mae_by_variation.png
```



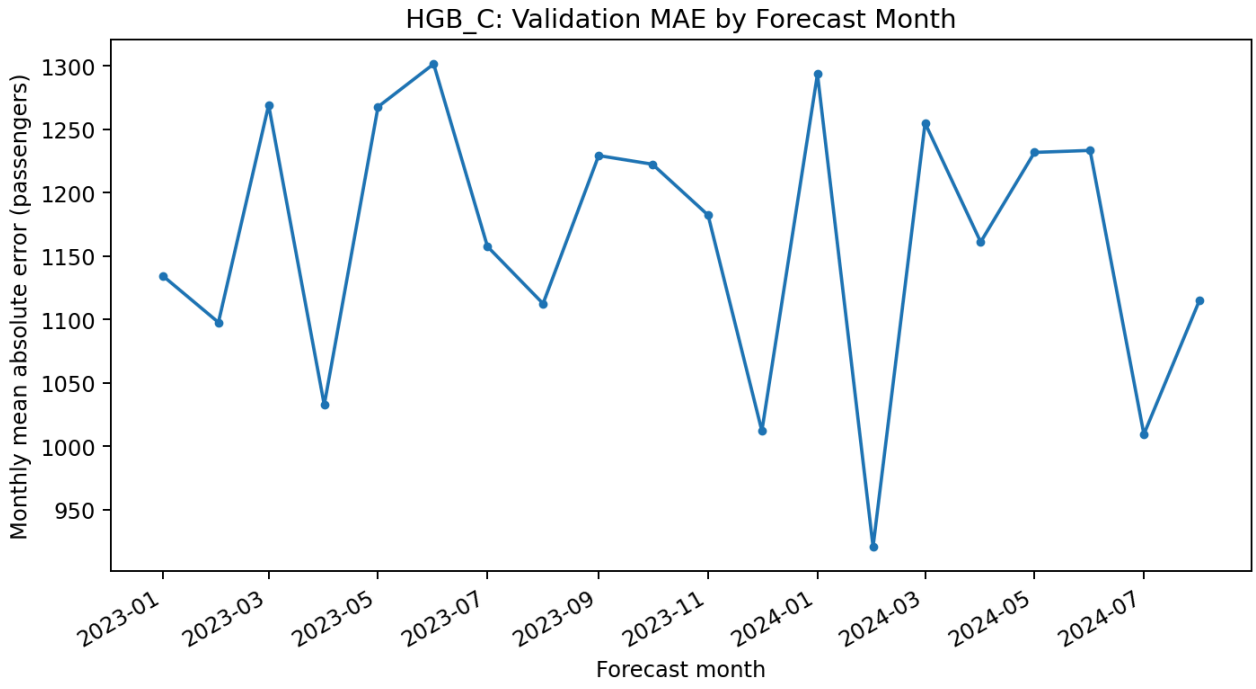
week7_figure_rmse_by_variation.png



week7_figure_validation_actual_vs_predicted.png



week7_figure_monthly_validation_mae.png



8. Reproducibility and completion audit

Only the three predetermined variations are tested. The existing chronological training/validation split remains unchanged, PCA is training-fitted, all predictions are nonnegative for evaluation, and **no test performance has been obtained**.

In [11]:
```
assert (TABLES/'week7_three_model_comparison.csv').is_file()
assert (TABLES/'week7_winner_summary.json').is_file()
assert (REPORTS/'week7_verified_results_for_report.md').is_file()
assert (ROOT/'Models'/'week7'/'week7_selected_hist_gradient_boosting.joblib').is_file()
for p in ['Training_MAE','Training_RMSE','Training_R2','Validation_MAE','Validation_RMSE','Validation_R2']:
    assert np.isfinite(results.comparison[p]).all()
assert (results.comparison[['Training_MAE','Training_RMSE','Validation_MAE','Validation_RMSE']] >= 0).all().all()
print('All Week 7 checks passed.')
print('===== WEEK 7 END =====')
```

```
All Week 7 checks passed.
===== WEEK 7 END =====
```